Insight

40

```
def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B log.record(id, amount) }
} } }
```

```
update(src, +100)
```

```
update(src, -100)
```

Program structure including nesting should effect possible behaviour orderings

```
when(dst) { C print(dst.balance) }

when(src) { D print(src.balance) }
```

Update path (`src` then `dst`)

Diagnostic path (`dst` then `src`)

```
when(src) { A  src.balance += 10; }

when(dst) { B  dst.balance += 10; }
```

Implementation details such as scheduling and runtime structures should not

Insight

```
def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B log.record(id, amount) }
} } }
```

`update(src, +100)`

`update(src, -100)`

Program structure including nesting should effect possible behaviour orderings

Update path (`src` then `dst`)

```
when(src) { A src.balance += 10; }
when(dst) { B dst.balance += 10; }
```

Diagnostic path (`dst` then `src`)

```
when(dst) { C print(dst.balance) }
when(src) { D print(src.balance) }
```

Implementation details such as scheduling and runtime structures should not

Why does this matter

- How can we reason about the validity of program optimisations?

```
when(busy, idle) { A
  val r = busy.use();
  idle.use(r)
  when(log) { B }
  idle.use(r)
}
when(busy) { C
  when(log) { D }
}
```

Can we release cowns early?

```
when(src) { E
  when(dst) { F }
}
when(dst) { G }
```

```
when(dst) { G }
when(src) { E
  when(dst) { F }
}
```

Can we reorder behaviours?

- What does it mean to say that an operational semantics or implementation is correct?